

WHATSAPP BOT: CASE CREATION FLOW SPECIFICATION

High-Level Flow Design, Data Intake Scenarios, and Backend API Integration Mapping

Document Reference	LAW-WA-BOT-SPEC-v1.0	Target API Platform	Meta Cloud API (WhatsApp Business)
System Context	LawOnCall Lead Intake Service	Status	Architectural Draft

1. Introduction & Architectural Objective

To reduce friction for clients seeking legal assistance, LawOnCall is introducing an automated conversational WhatsApp Bot using the **Meta Cloud API**. The bot operates as a guided intake form that extracts customer details and automatically registers them as a **Lead** in our database, preparing them for an instant consultation callback.

2. Customer Information Required

To successfully register a legal case in our database schema, the bot must collect these parameters:

- Name: The user's full legal name (for case registration).
- Phone Number: Extracted **automatically** from the incoming WhatsApp sender metadata (the user does not need to type this).
- Legal Category: The area of law (must match one of our supported classes: **Family** , **Corporate** , **Criminal** , **Property** , Or **Labour**).
- City: The user's location (to route to locally registered advocates).
- Problem Description: A short text describing the issue (the "case brief").
- Preferred Callback Time: The slot preference (e.g. **Immediate** , **Evening (5-8 PM)** , Or **Tomorrow Morning**).

3. Conversational Flows & Scenarios

Here are the step-by-step user input/output conversational designs for various user scenarios.

● Scenario A: Normal Success Flow (First-Time User)

- **User:** Writes "Hi" or clicks your WhatsApp link.
- **Bot:** "Welcome to LawOnCall. Let's create your consultation request. Please reply with your ****Full Name****."
- **User:** Inputs "Aman Agarwal".
- **Bot:** Displays an ****Interactive List Message**** with buttons:
"Please select your legal category:"
[Family Law] [Property Dispute] [Corporate/Business] [Other]
- **User:** Clicks ****[Property Dispute]****.
- **Bot:** "Thank you. Which ****City**** are you located in?"
- **User:** Inputs "Delhi".
- **Bot:** "Please describe your legal issue in a single message."
- **User:** Inputs "Landlord is refusing to return my security deposit of 50,000 INR."
- **Bot:** Displays callback slot buttons:
"When would you like our legal advocate to call you?"
[Immediate (within 15 mins)] [Evening (5–8 PM)] [Tomorrow]
- **User:** Clicks ****[Immediate]****.
- **Bot:** "Perfect! Your case has been registered. A verified advocate will call you on +91xxxxxxxxx shortly. You can track your case on: <https://lawoncall.in>"

● Scenario B: Category / Input Validation Error

- **User:** Is at the category step, but instead of clicking the buttons, they type a random message:
"Help me please!"
- **Bot:** Detects invalid input.
"I didn't quite catch that. Please select one of the legal categories below using the buttons:"
[Family Law] [Property Dispute] [Corporate/Business] [Other]

● Scenario C: Drop-off Reminder (SLA Flow)

If the user stops replying midway (e.g., stops at the problem description step):

- **System (After 30 minutes of inactivity):** Sends an automated notification:
"Hi Aman, we noticed you were creating a consultation request. Would you like to complete it now?"
[Yes, Continue] [Cancel Request]
- **User:** Clicks ****[Yes, Continue]****. The bot resumes from the step they dropped off.

4. Backend API Integration Specifications

To integrate this bot with the LawOnCall monorepo backend, the developer must implement a webhook receiver endpoint.

4.1. Meta Webhook URL: `POST /api/whatsapp/webhook`

Your backend must verify the webhook URL challenge from Meta (`GET`) and process the incoming messages (`POST`). The incoming metadata contains the sender's phone number and the message text or button payload.

```
// Example Incoming Message Payload from Meta Cloud API
{
  "object": "whatsapp_business_account",
  "entry": [{
    "id": "WHATSAPP_BUSINESS_ACCOUNT_ID",
    "changes": [{
      "value": {
        "messaging_product": "whatsapp",
        "metadata": {
          "display_phone_number": "15550000000",
          "phone_number_id": "PHONE_NUMBER_ID"
        },
        "contacts": [{
          "profile": { "name": "Aman Agarwal" },
          "wa_id": "919876543210" // This is the user's phone number!
        }],
        "messages": [{
          "from": "919876543210",
          "id": "wamid.HBgLOTE5ODc2NTQzMjEwFQIAERgSQjM...",
          "timestamp": "1672531199",
          "text": { "body": "Delhi" },
          "type": "text"
        }]
      },
      "field": "messages"
    }]
  }]
}
```

4.2. Database & State Management

To track where the user is in the intake flow (since HTTP requests are stateless), the backend must store a temporary user state record (e.g. in Redis or a new table `WhatsAppSession`):

```
model WhatsAppSession {  
    id          String    @id @default(uuid())  
    phoneNumber  String    @unique  
    currentStep  String    // "AWAITING_NAME", "AWAITING_CATEGORY", "AWAITING_CITY", "AWAITING_DESC"  
    tempName     String?  
    tempCategory String?  
    tempCity     String?  
    tempDesc     String?  
    createdAt    DateTime @default(now())  
    updatedAt    DateTime @updatedAt  
}
```

4.3. Creating the Lead

Once the final slot is selected, the webhook handler deletes the `WhatsAppSession` record and makes an internal call to the Lead creation API: `POST /api/leads/create`, passing the collected metadata.